Actividad 3: Enfoque Modular y Diseño Top-Down – Cálculo del Área de un Círculo

Ian Quiroga

Santino Laudati

Dante Iglesias

E.E.S.T. N.° 1 "Eduardo Ader" – Vicente López

Curso: Laboratorio de Programación 5° 3° (Grupo G03)

Profesor: Mansilla Muñoz York E.

Fecha: 04/11/2026

Versión	Fecha	Descripción	Autores
v1.0.0	04/11/2026	Baseline Aprobado	I. Quiroga, S. Laudati, D. Iglesias

Introducción

El objetivo de esta actividad fue aplicar el enfoque estructurado y *Top-Down* (de lo general a lo particular) para construir un programa modular en C++ que calcula el área de un círculo con la fórmula $A = \pi \cdot r^2$. En lugar de escribir toda la lógica dentro de `main()`, el cálculo se separó en una función propia, `calcularAreaCirculo()`, declarada mediante un prototipo antes del programa principal.

El programa además valida el dato ingresado con un filtro de consistencia (el radio debe ser positivo) y, como bonus de gamificación, dibuja en la consola una diana de tiro cuyo tamaño depende del radio. En este informe se explican los bloques completados, el diagrama de flujo, las pruebas y las respuestas al cuestionario técnico.

Desarrollo

Integrantes y tareas realizadas

Tabla 1

Distribución de tareas del grupo

Integrante	Tareas realizadas
Ian Quiroga	Completó los bloques 2, 3 y 4 de `src/main.cpp` (llamada, salida y desarrollo de la función), agregó la validación de datos no numéricos, compiló y probó el programa.
Santino Laudati	Redacción del informe APA v7 y de las respuestas del cuestionario; explicación del diseño Top-Down.
Dante Iglesias	Diagrama de flujo, capturas de compilación y ejecución, README, manual del programador y repositorio Git.

Diseño Top-Down del problema

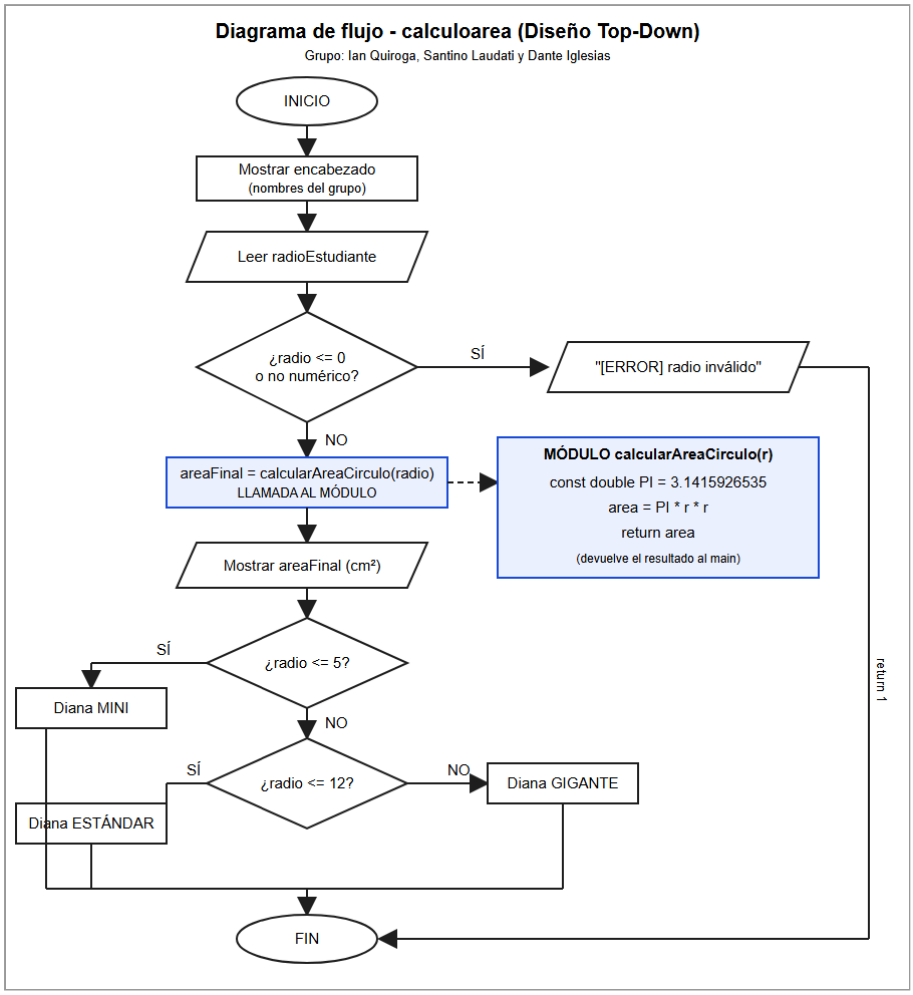
El problema general ("calcular y mostrar el área de una diana") se dividió en módulos más pequeños:

1. **Entrada:** pedir el radio al usuario.
2. **Validación:** filtro de consistencia que rechaza radios menores o iguales a cero.
3. **Cálculo:** módulo independiente `calcularAreaCirculo(r)` que aplica la fórmula y devuelve el resultado.
4. **Salida:** mostrar el área y dibujar la diana según el tamaño.

Esta división permite encontrar errores más rápido, reutilizar la función de cálculo en otros programas y repartir el trabajo entre los integrantes.

Figura 1

Diagrama de flujo del programa modular calculoarea



Bloques completados del código

Bloque 2 – Llamada a la función. Se invoca el módulo pasándole el radio y el valor que devuelve se guarda en `areaFinal`:

```
double areaFinal = calcularAreaCirculo(radioEstudiante);
```

Bloque 3 – Salida de datos. Se muestra el radio y el área calculada:

```
cout << "\n[RESULTADO] Radio ingresado: " << radioEstudiante << " cm" << endl;
cout << "[RESULTADO] El area del circulo es: " << areaFinal << " cm2" << endl;
```

Bloque 4 – Desarrollo de la función. Se utiliza una variable local `area` y la constante `PI` bloqueada con `const`:

```
double calcularAreaCirculo(double r) {
    const double PI = 3.1415926535;
    double area = PI * r * r;   // variable local
    return area;                // se devuelve el resultado al main()
}
```

Además, el filtro de consistencia se amplió con `cin.fail()` para que el programa también rechace letras u otros datos no numéricos, que de lo contrario dejarían el radio en 0 sin mostrar un mensaje claro.

Pruebas y evidencias

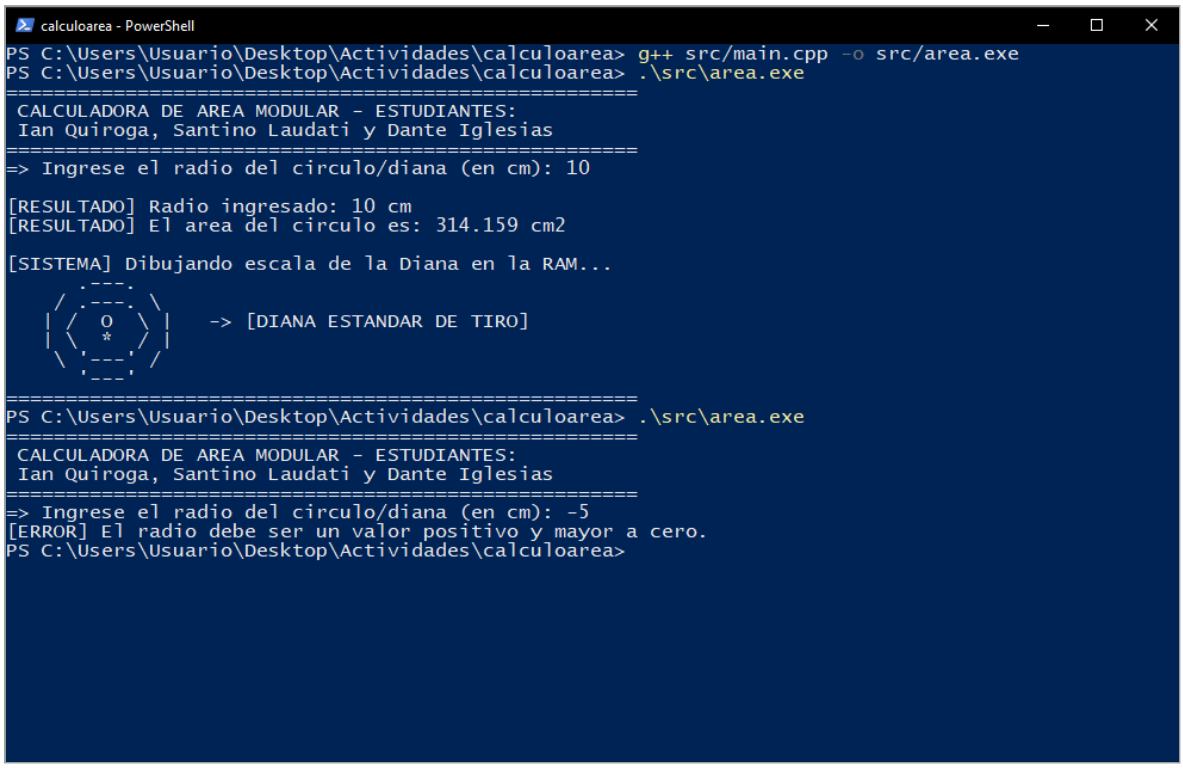
Tabla 2

Casos de prueba

Radio ingresado	Área esperada (cm²)	Diana	Resultado
3	28.2743	Mini / compacta	Correcto
10	314.159	Estándar de tiro	Correcto
20	1256.64	Gigante de cobertura	Correcto
0	—	Mensaje de error	Correcto
-5	—	Mensaje de error	Correcto
abc	—	Mensaje de error	Correcto

Figura 2

Compilación con g++ y ejecución con radio 10 (válido) y -5 (rechazado por el filtro de consistencia)



Comandos utilizados desde la raíz de `calculoarea`:

```
g++ src/main.cpp -o src/area.exe
.\src\area.exe
```

Cuestionario técnico de control

Pregunta 1 – El prototipo de la función. Si borráramos la línea `double calcularAreaCirculo(double radio);`, el compilador, que lee el archivo de arriba hacia abajo, llegaría a la llamada dentro de `main()` sin conocer todavía esa función: no sabría qué tipo de dato devuelve ni qué parámetros recibe. Por eso arrojaría el error *'calcularAreaCirculo' was not declared in this scope* y no generaría el ejecutable. El prototipo funciona como un aviso previo de la "firma" de la función.

Pregunta 2 – El filtro de consistencia. Un círculo con radio 0 no existe: es solo un punto y su área sería 0. Un radio negativo no tiene sentido físico, porque una distancia no puede ser negativa; sin embargo, como la fórmula eleva el radio al cuadrado, $(-5)^2$ da 25 y el programa mostraría un área positiva de 78.54 cm^2 que es falsa. Controlar esto es

importante para que el sistema nunca procese datos imposibles ni entregue resultados engañosos, algo crítico en software real de medición o ingeniería.

Pregunta 3 – Proteger constantes en RAM. Declarar un valor como `const` hace que el compilador impida modificarlo: si alguien escribe por error `PI = 5.5;`, la compilación se detiene antes de que el programa llegue a ejecutarse. En un sistema crítico, como el software de un avión o de un equipo médico, un valor de referencia alterado por accidente podría provocar cálculos erróneos con consecuencias graves. Con `const` el error se detecta en la etapa de compilación, que es mucho más segura y barata que descubrirlo en funcionamiento.

Conclusión

Esta actividad nos mostró la ventaja de pensar un programa de lo general a lo particular. Al separar el cálculo en su propia función, `main()` quedó más corto y fácil de leer, y la función `calcularAreaCirculo()` podría reutilizarse en cualquier otro programa sin cambios.

También entendimos el rol de cada parte de una función en C++: el prototipo avisa al compilador, la llamada envía los datos, el desarrollo realiza el cálculo con variables locales y el `return` devuelve el resultado. El filtro de consistencia y el uso de `const` nos hicieron ver que un buen programador no solo hace que el código funcione, sino que también lo protege de errores del usuario y de errores propios.

Referencias

cplusplus.com. (s.f.). *Functions*. https://cplusplus.com/doc/tutorial/functions/

Mansilla Muñoz, Y. E. (2026). *Actividad 3: Enfoque modular y diseño top-down* [Material de clase]. E.E.S.T. N.° 1 "Eduardo Ader".

Microsoft. (2024). *Funciones (C++)*. Microsoft Learn. https://learn.microsoft.com/es-es/cpp/cpp/functions-cpp

Microsoft. (2024). *const (C++)*. Microsoft Learn. https://learn.microsoft.com/es-es/cpp/cpp/const-cpp

Quiroga, I., Laudati, S., & Iglesias, D. (2026). *calculoarea* [Repositorio de código fuente]. GitHub. https://github.com/pachusUwu/calculoarea